Cera Functional Language - 1.0

Isaac Honeyman

June 30, 2026

Abstract

This document outlines the architectural pipeline, implementation phases, and execution mechanics of the Cera custom functional programming language compiler and runtime system.

Contents

1 Core Design Ideas

1.1 Principles

- **True Functionality:** The language will be a true functional language, with 0 side effects (excluding I/O) or mutable state.
- **OOP Style Syntax:** The syntax must be easily readable and writeable, by those familar with OOP languages (Java, C#, python in particular).
- **First-Class Functions:** Functions are first-class citizens, supporting higher-order programming and closures.
- **Evaluation Strategy:** The language enforces call-by-value (strict) evaluation. deferred execution and lazy evaluation can be simulated via use of `unit` thunks (e.g. `unit -> g`)
- **Type System:** Explicit typing for functions, and variables use local type inference.
- **Memory Model:** Reference counting system.
- **Simplisitc, Streamlined Design:** While all programming problems can be solved with near infinite solutions, the language should not offer many similar, slightly differing methods of completing a task, and focus on a simple set of syntax and keywords (see `lazy` in excluded features).
- **Backwards Compatibility:** Old code should work in newer Cera versions.

1.2 General Conventions

- **Variables:** variables must use camelCase, and type annotation should be used if not obvious from the context.
- **Functions:** function names must also use camelCase, and must have an explicit type signature, this includes built-in functions.
- **Types:** all built-in types feature a main type signature, that will be lowercase, with pascal case constructors.

1.3 Tech Stack

- **Compiler:** The bytecode compiler will be written in C#, as it's a familar language with easy parsing tools.
- **Virtual Machine:** The virtual machine will be written in C, as it is a low-level language with easy memory management and high performance.
- **Build System:** We shall use a bash script to build the compiler and virtual machine, and to run tests.

2 Language Syntax

2.1 Types

The language base type set will be extremely minimal, all using lowercase names, with the following types and type 'extensions':

- int: 64-bit signed integer (2's complement)
- float: 64-bit IEEE 754 floating point number

- bool: 1-bit boolean value (true or false)
- char: 32-bit Unicode character
- unit: A type representing the absence of a meaningful mathematical value, used for side effects, and delayed evaluation. It has exactly one valid value, written as `()`.
- **list:** linked list of values of a single type (OCaml-style)
- **arr:** fixed-length contiguous array, assignable once upon creation, addressable via the built in get keyword.
- **tup:** fixed-length tuple of values of potentially different types, note this is not a keyword unlike other extensions, and type definition must be wrapped in brackets to prevent ambiguity.

Note that in the following snippets, we can chain together extensions to create more complex types, aswell as the syntax sugar for a char list.

Cera Language

```
def foo() : unit = {
    var x: int = 5;
    var y: int list = [1, 2, 3, 4];
    var z: char list list = [['a', 'b'], ['c', 'd']];
    var s: char list = "hello world";
    var w: bool arr = arr[true, false, true];
    var u: (int * int * int) = (1, 2, 3);
    var f: int -> int -> int =
      func (x: int, y: int) : int = x + y;
    ();
}
```

2.2 Variable Declarations and Scoping

Variables in Cera are immutable bindings, explicitly or implicitly typed upon creation. Because the language enforces zero mutable state, a variable's value can never be altered after it is bound.

- **Bindings:** Variables are declared using the `var` keyword. If a type annotation is omitted, the compiler utilizes local type inference to determine the type from the evaluated expression.
- **Shadowing:** Cera supports variable shadowing within the same scope. Re-declaring a variable with an existing identifier creates a completely new memory binding that obscures the original identifier for the remainder of the scope, preserving immutability.
- **Strings:** The language does not have a distinct string primitive; syntaxtic sugar can be used, and strings are evaluated syntactically as a `char list`.

Variable Binding and Shadowing

```
def foo() : unit = {
    var x: int = 5;
    // x is mathematically bound to 5

    var x: int = 10;
    // Legal shadowing. the old x is inaccessible.
```

```
    var greeting = ['H', 'i']; // char list
    var greeting2 = "Hi"; // char list

    ();
}
```

2.3 Operators and Expressions

In Cera, all operations are evaluated strictly by default. The language supports standard arithmetic, relational, and logical operators. Because Cera is a strictly typed language, operators do not implicitly cast between types (e.g., an `int` cannot be added directly to a `float` without explicit conversion). This process shall be done by built-in function calls.

- **Arithmetic:** Standard binary operators for numeric types: `+`, `-`, `*`, `/`, and `%` (modulo) (integer and float).
- **Bitwise:** Bitwise operators: `&`, `|`, `^`, `~`, `<<`, and `>>` (integer).
- **Relational:** Comparison operators returning a `bool`: `==`, `!=`, `<`, `>`, `<=`, `>=`. Structural equality is evaluated for composite types like tuples and lists (only == and !=).
- **Logical:** Boolean operators: `&&` (logical AND), `||` (logical OR), and `!` (logical NOT). These support short-circuit evaluation.
- **Concatenation and Lists:** The `::` operator is used to prepend an element to a list. The use of a `concat` function will concatenate two lists/arrays of the same type.
- **Conditional:** Use of the `?` and `:` operators for inline conditional expressions (ternary operator).

Expressions and Operators

```
def foo() : unit = {
    var a: int = 10 + 5;
    var b: float = 3.14 * 2.0;
    var isValid: bool = (a > 20) && (b < 10.0); // false
    var sameTuple: bool = (1, 2) == (1, 2); // true
    var combined: int list = concat([1, 2], (3::[4, 5]));
    var result: int = isValid ? 100 : a > 20 ? 200 : 300; // 300
    ();
}
```

2.4 Control Flow

Because Cera enforces zero mutable state, standard control flow statements are replaced by **control flow expressions**. Every conditional construct evaluates to a return value. To maintain type safety, the compiler strictly enforces that all branches within a single control flow construct evaluate to the exact same type.

- **Expression Blocks:** Standard `if/else` and `else if` blocks that evaluate to the value of their final expression. The `return` keyword is not used; the last evaluated expression in the block is returned implicitly.
- **Ternary Operator:** The inline `?  :` operator is supported for concise, single-line conditional assignments. As noted in the operators section, chained ternary operators are right-associative.

- **Pattern Matching:** The `switch` construct replaces traditional statement-based switch blocks. It is the primary mechanism for structurally unpacking tuples, lists, and complex types. Branches are defined using the `->` operator, and the compiler enforces exhaustive matching.

Control Flow Expressions

```
def foo() : unit = {
    // 1. Ternary Operator
    var status: int = (health > 50) ? 1 : 0;

    // 2. Expression Block
    var multiplier: float = if (status == 1) {
        var bonus: float = 1.2;
        baseRate * bonus;
    } else if (status == 0) {
        baseRate * 0.5;
    } else {
        baseRate;
    };

    // 3. Pattern Matching via Switch
    var location: int = switch (coordinates) {
        (0, 0) -> 1,
        (x, 0) -> 2,
        (0, y) -> 3,
        (x, y) -> 4,
    }

    ();
}
```

2.5 Functions

Functions in Cera are first-class citizens. Because the language enforces zero mutable state, there are no traditional looping constructs (e.g., `for` or `while` loops); all iteration is achieved via recursion.

- **Declarations:** Named functions are declared using the `def` keyword. All parameters and the return type must be explicitly annotated. The body of the function is an expression block, and the final evaluated expression is implicitly returned.
- **Higher-Order Functions:** Functions can be passed as arguments to other functions and returned as values, utilizing the arrow typing syntax (e.g., `int -> int`).
- **Recursion:** Functions in Cera are recursive by default, allowing a function to call itself within its own body without additional keywords.
- **Anonymous Functions (Lambdas):** Lambda functions are defined using the `func` keyword, followed by explicitly typed parameters, a strictly required return type annotation, and either an expression block or a single inline expression.
- **Generic Functions:** Cera supports generic functions by appending type parameters in angle brackets directly after the function name (e.g., `def name<g1, g2>`). These generic types can be used throughout the function signature and body.

- **Scoping and Shadowing:** All named functions (declared via `def`) must reside in the global scope and possess strictly unique identifiers. Function shadowing is not permitted, and named inner functions are forbidden to maintain a flat AST. Localized closures must be handled via anonymous lambda functions.
- **Function Application:** Cera does not support automatic partial function application (currying). A function call must supply the exact number of arguments specified in its signature. Developers requiring partially applied functions must explicitly construct them using lambdas.

Function Declarations

```
def factorial(n: int): int = {
    if (n <= 1) {
        1;
    } else {
        n * factorial(n - 1);
    }
}

def applyTwice(f: int -> int, x: int): int = {
    f(f(x));
}

// Lambda with an inline expression
applyTwice(func (x: int): int = x + 3, 5); // returns 11

// Generic function using pattern matching
def head<g>(x: g list, default: g): g = {
    switch (x) {
        [] -> default,
        (h::_) -> h
    }
}
```

2.6 Custom Types (Algebraic Data Types)

Cera supports the creation of custom Algebraic Data Types (ADTs). To align with the built-in primitive types, user-defined type identifiers are strictly lowercase, while their data constructors must be capitalized (PascalCase). This guarantees the lexer can instantly distinguish between a type signature and a concrete data constructor.

- **Sum Types (Variants):** Defined using the `type` keyword. Constructors are separated by the `|` (OR) operator. If a constructor carries a data payload, the `:` operator is used to specify the underlying type.
- **Recursive Types:** Types can reference themselves within their own definitions, allowing for recursive data structures like trees and streams.
- **Generic Types:** Type parameters are declared using angle brackets (e.g., `<g>`) and passed consistently to recursive calls.
- **Built In:** The `option` and `result` types are built-in ADTs used for error handling and optional values, as detailed in the Error Handling section.

Algebraic Data Types

```
// Standard ADT with the
type shape = Circle: float | Rect: (float * float);

// Generic, recursive ADT
type stream<g> = Nil | Cons: (g * stream<g>);

// Utilizing the custom type in a switch expression
def area(s: shape): float = {
    switch (s) {
        Circle(r) -> 3.14159 * r * r,
        Rect(w, h) -> w * h
    }
}
```

2.7 Error Handling

To maintain absolute mathematical purity and predictable control flow, Cera completely omits `try/catch` exception blocks and the concept of `null` references. Instead, functions that can fail or return no value must make that failure explicit in their type signature using built-in Algebraic Data Types.

- **The Option Type:** Used when a function might legitimately return nothing. It is defined as `type option<g> = None | Some: g`.
- **The Result Type:** Used when a function can fail and needs to return an error message or code. It is defined as `type result<v, e> = Ok: v | Error: e`.

These types force the programmer to handle both the success and failure states using `switch` expressions, guaranteeing that unhandled exceptions cannot crash the VM.

Error Handling via ADTs

```
def divide(a: float, b: float): result<float, string> = {
    if (b == 0.0) {
        Error("Division by zero");
    } else {
        Ok(a / b);
    }
}
```

2.8 Program Entry and I/O

Cera enforces zero mutable state and forbids user-defined side effects. However, a program must interact with the host system to be useful. This is achieved through controlled, built-in impure functions (intrinsics) and a strictly defined execution entry point.

- **Impure Intrinsics:** I/O operations (such as `readFile`, or network requests) are provided exclusively as compiler built-ins. Users cannot author their own impure functions from scratch within Cera.
- **The Entry Point:** Program execution begins strictly at the `entry` function. This function serves as the bridge between the host operating system and the pure functional runtime. It takes an array of command-line arguments and must evaluate to an integer representing the system exit code (where 0 indicates success).

Program Entry

```
def entry(args: char list arr): int = {
    // Built-in impure function call
    outLst("hello world");

    var status: int = runCoreLogic();
    status; // Evaluates to the exit code
}
```

2.9 Deferred Evaluation

Cera evaluates all expressions strictly (call-by-value) by default. However, to support infinite data structures and deferred execution, one may use unit thunks.

Infinite Streams

```
type stream<g> = Nil | Cons: (g * unit -> stream<g>);

def NaturalNumbers(x : int): stream<int> = {
    Cons(x, func (y : unit) :
        stream<int> = NaturalNumbers(x+1));
}
```

2.10 Miscellaneous Syntax

- **Comments:** Cera supports single-line comments using the standard `//` syntax. The lexer ignores all text from the `//` token to the end of the line. Block comments can be implemented using `/*` to start and `*/` to end, allowing for multi-line comments that can be nested.
- **Multifile Programs:** Cera takes a c style approch to simply multifile programs, to use another file, a cera file must being with the syntax `import "FileName";` - these are then recursively dealt with, (keeping track of imported files to prevent cyclic loops), and are treated as essentialy one large file. Note due to this, types and functions identifiers must be unique across all files.

2.11 Summary

To assist in the lexical analysis phase, the following is an exhaustive list of reserved keywords in the Cera language. These identifiers are strictly reserved by the grammar and cannot be used as variable, function, or custom type names.

- **Declaration & Binding:** `var`, `def`, `func`, `type`
- **Control Flow:** `if`, `else`, `switch`, `import`
- **Primitive Types:** `int`, `float`, `bool`, `char`, `list`, `arr`, `unit`
- **Boolean Literals:** `true`, `false`

Note: Identifiers such as `concat`, `outLst`, and `entry` are considered built-in intrinsic functions rather than reserved lexical keywords.

Next is a list of built in ADT:

- **Option:** `type option<g> = None | Some:  g`

- **Result:** `type result<v, e> = Ok: v | Error: e`

Finally the list of built in functions:

- **Get:** `get<g>(target: g arr, index: int) : option<g>` (0 based array index).
- **Arr Length:** `arrLength<g>(array: g arr) : int`
- **Concat:** `concat<g>(left: g list, right: g list) : g list`
- **Arr Concat:** `arrConcat<g>(left: g arr, right: g arr) : g arr`
- **Out:** `out(output: char list) : unit`
- **In:** `in() : char list`
- **Read:** `read(path: char list) : result<char list, char list>`
- **Write:** `write(path: char list, content: char list) : result<unit, char list>`
- **Int To Float:** `intToFloat(x: int) : float`
- **Float To Int:** `floatToInt(x : float) : int` (truncates)
- **Char To Int:** `charToInt(c : char) : int`
- **Int To Char:** `intToChar(x : int) : char`
- **Int To Chars:** `intToChars(x : int) : char list`
- **Float To Chars:** `floatToChars(x : float) : char list`
- **Bool To Chars:** `boolToChars(x : bool) : char list`
- **Chars To Int:** `charsToInt(str : char list) : option<int>`
- **Chars To Float:** `charsToFloat(str : char list) : option<float>`
- **Arr To List:** `arrToList<g>(array: g arr) : g list`
- **List To Arr:** `listToArr<g>(lst: g list) : g arr`

3 Lexical Analysis (Tokenisation)

The lexical analysis phase operates as a Deterministic Finite Automaton (DFA), consuming the raw UTF-8 character stream of the source file and segmenting it into a linear sequence of logical `Token` structures. This process strips away non-semantic formatting, preparing the data for syntax analysis.

3.1 The Token Structure

Each emitted token encapsulates three critical pieces of data to ensure both accurate parsing and robust error reporting:

- **Tag (Type):** An enumerated value classifying the token (e.g., `IDENTIFIER`, `INT_LIT`, `DEF_KW`, `PLUS_OP`).
- **Lexeme:** The concrete string of characters that formed the token.
- **Location Metadata:** The line and column index marking the token's origin, strictly utilized for traceback generation during semantic or syntactic failures.

3.2 Lexical Categories and Regular Expressions

The DFA categorizes lexemes based on the following rules and regular expressions. The scanner employs the "maximal munch" principle, always matching the longest possible valid lexeme (e.g., reading `<=` as a single relational operator rather than a `<` followed by an `=`).

- **Identifiers:** Variables and functions map to `[a-z][a-zA-Z0-9_]*`. Custom type constructors must begin with a capital letter, mapping to `[A-Z][a-zA-Z0-9_]*`.
- **Keywords:** Reserved language keywords (e.g., `var`, `def`, `switch`) are initially scanned as standard identifiers. Before emission, they are checked against a static hash map and re-tagged with their specific keyword enumeration to achieve $O(1)$ lookup times.
- **Literals:**
  - *Integer:* `[0-9]+`
  - *Float:* `[0-9]+.[0-9]+`
  - *Character:* `'[^']'`
- **Operators and Punctuation:** Single-character punctuation (e.g., `+`, `*`, `{`, `;`) transitions immediately to an accept state. Potential multi-character operators (e.g., `==`, `->`, `::`) require a single character of lookahead to determine the correct transition.

The following is a full list of tokens used within the lexer:

- **Declaration:** `Var`, `Def`, `Func`, `Type`
- **Control Flow:** `If`, `Else`, `Switch`, `Import`
- **Primitive Types:** `Int`, `Float`, `Bool`, `Char`, `List`, `Arr`, `Unit`, `WildCard`
- **Literals:** `True`, `False`, `Identifier`, `Constructor`, `IntLiteral`, `FloatLiteral`, `CharLiteral`, `StringLiteral`
- **Flow:** `LBracket`, `RBracket`, `LBRace`, `RBrace`, `LPar`, `RPar`
- **Structural:** `Equal`, `SemiColon`, `Comma`
- **Operators:** `Plus`, `Minus`, `Star`, `Slash`, `Mod`, `BitAnd`, `BitXor`, `BitNot`, `LShift`, `RShift`, `EqualEqual`, `NotEqual`, `Lesser`, `LesserEqual`, `Greater` `GreaterEqual`, `And`, `Or`, `Not`, `ColonColon` `Question`, `Colon`, `Pipe`, `Arrow`,
- **Special:** `None`

Note: The `None` token type is simply used for errors, and represents the absence of a token.

3.3 Whitespace and Comment Handling

Whitespace characters (spaces, tabs, carriage returns, and newlines) serve strictly as lexeme delimiters. When the DFA encounters whitespace, it forces the completion and emission of the currently accumulating token. The whitespace itself is then immediately consumed and discarded; no token is generated.

Similarly, single-line (`//`) and block (`/* ... */`) comments transition the scanner into an absorption state, aggressively consuming characters until the termination sequence is reached, emitting nothing to the parser stream.

3.4 Import Handling

As will be seen in the next section (Parsing), the import keyword and it's syntax `import "filename"` are not present. This is an intentional choice to prioritise simplicity of our compilation process, and is handled instead in the code via an intermediate `ImportResolver` stage, which searches the top of lexed files for imports, such the entire included codebase can be parsed as one unit.

4 Syntax Analysis (Parsing)

4.1 Notation Guide

The syntax is formally specified using Extended Backus-Naur Form (EBNF). To avoid ambiguity between the meta-syntax and the language's own lexical tokens, the following typographic conventions are strictly applied:

- ⟨Rule⟩: Angle brackets denote a **non-terminal** production rule.
- **keyword**: Bold, monospace text denotes a **terminal** symbol (a concrete lexical token, keyword, or required punctuation like (or }).
- ::=: The definition operator, read as "is defined as".
- |: The alternation operator, denoting a choice between derivations ("or").
- [...]: Square brackets enclose an **optional** component (zero or one occurrences).
- (...): Standard mathematical parentheses denote **logical grouping** of meta-symbols. They are *not* literal characters in the source code.
- x^*: The Kleene star suffix indicates **zero or more** consecutive repetitions of x.
- x^+: The Kleene plus suffix indicates **one or more** consecutive repetitions of x.

4.2 Context-Free Grammar (CFG)

The formal CFG for Cera is defined in Backus-Naur Form (BNF). To satisfy structural requirements for unambiguous derivations and Top-Down parsing without left recursion, the grammar stratifies expressions and types. The grammar enforces strict functional purity by restricting `var` bindings strictly to local expression blocks.

Are top level fundamental definitions:

⟨Program⟩ ::= (⟨FuncDecl⟩ | ⟨TypeDecl⟩)*
⟨FuncDecl⟩ ::= **def id** [⟨GenericDecl⟩] **(**[⟨Param⟩ (**,** ⟨Param⟩)*]**) :** ⟨Type⟩ = ⟨ExprBlock⟩
⟨TypeDecl⟩ ::= **type id** [⟨GenericDecl⟩] **=** ⟨ConDecl⟩ (| ⟨ConDecl⟩)***;**
⟨GenericDecl⟩ ::= <**id** (**,** **id**)*>
⟨Type⟩ ::= ⟨SuffixType⟩ [−> ⟨Type⟩]
⟨SuffixType⟩ ::= ⟨AtomType⟩ [**list** | **arr**]*
⟨AtomType⟩ ::= ⟨BaseType⟩ | ⟨TupleType⟩ | ⟨GenericType⟩ | **(**⟨Type⟩**)**
⟨BaseType⟩ ::= **int** | **float** | **bool** | **char** | **unit**
⟨TupleType⟩ ::= **(**⟨Type⟩ (***⟨Type⟩)+**)**
⟨GenericType⟩ ::= **id**<⟨Type⟩ (**,**⟨Type⟩)*>
⟨ConDecl⟩ ::= **con** [**:** ⟨Type⟩]
⟨ExprBlock⟩ ::= **{**⟨Stmt⟩* ⟨Expr⟩[**;**]**}**
⟨Stmt⟩ ::= ⟨VarDecl⟩ | ⟨Expr⟩**;**
⟨Param⟩ ::= **id :** ⟨Type⟩

Following are the expression definitions:

⟨Expr⟩ ::= ⟨BinOpChain⟩ [**?** ⟨Expr⟩ **:** ⟨Expr⟩]
⟨BinOpChain⟩ ::= ⟨UnaryExpr⟩ (⟨BinOp⟩ ⟨UnaryExpr⟩)*
⟨UnaryExpr⟩ ::= ⟨UnOp⟩ ⟨UnaryExpr⟩ | ⟨CallExpr⟩
⟨CallExpr⟩ ::= ⟨Primary⟩ (([⟨Expr⟩(**,** ⟨Expr⟩)*]))*
⟨Primary⟩ ::= ⟨Literal⟩ | **id** | **(**⟨Expr⟩**)** | ⟨IfExpr⟩ | ⟨SwitchExpr⟩ | ⟨Lambda⟩
⟨BinOp⟩ ::= + | **-** | ***** | / | % | **&** | | | ^ | << | >> | == | ! = | < | > | <= | >= | **&&** | || | **::**
⟨UnOp⟩ ::= **!** |∼| **-**
⟨IfExpr⟩ ::= **if (**⟨Expr⟩**)** ⟨ExprBlock⟩ (**else if (**⟨Expr⟩**)** ⟨ExprBlock⟩)* [**else** ⟨ExprBlock⟩]
⟨SwitchExpr⟩ ::= **switch (**⟨Expr⟩**) {**⟨PatternMatch⟩ (**,** ⟨PatternMatch⟩)***}**
⟨PatternMatch⟩ ::= ⟨Pattern⟩ −> ⟨Expr⟩
⟨Pattern⟩ ::= ⟨Literal⟩ | **id** | _ | **(**⟨Pattern⟩ (**,** ⟨Pattern⟩)+**)** | [[⟨Pattern⟩ (**,** ⟨Pattern⟩)*]]
| **arr** [[⟨Pattern⟩ (**,** ⟨Pattern⟩)*]] | **(**⟨Pattern⟩ **::** ⟨Pattern⟩**)**
| **con**[**(**⟨Pattern⟩ (**,** ⟨Pattern⟩)***)**]
⟨Lambda⟩ ::= **func (**[⟨Param⟩ (**,** ⟨Param⟩)*]**) :** ⟨Type⟩ **=** (⟨Expr⟩ | ⟨ExprBlock⟩)
⟨Literal⟩ ::= **int_lit** | **float_lit** | **char_lit** | **str_lit** | **true** | **false** | **()** | ⟨ListLit⟩ | ⟨ArrLit⟩ | ⟨TupleLit⟩
⟨ListLit⟩ ::= [[⟨Expr⟩ (**,** ⟨Expr⟩)*]]
⟨ArrLit⟩ ::= **arr** [[⟨Expr⟩ (**,** ⟨Expr⟩)*]]
⟨TupleLit⟩ ::= **(**⟨Expr⟩ (**,** ⟨Expr⟩)+**)**

Note: That **id** and **con** refer to identifier and constructor tokens respectively.

4.3 Lexical Ambiguity Resolution

Because the lexical analyser adheres strictly to the "maximal munch" principle, the character sequence `>>` is always greedily tokenised as a single right-shift operator (`>>`), rather than two distinct `>` tokens. This creates a standard structural collision in $LL(k)$ parsers when evaluating nested generic type closures (e.g., `option<stream<g>>`).

To resolve this without breaking lexical encapsulation or resorting to a "lexer hack", the ambiguity is handled entirely within the syntax analyser. When the parser is evaluating a generic type signature and expects a closing bracket `>`, but instead encounters a right-shift `>>` token, it consumes the operator, logically applies the first `>`, and modifies the token stream (or internal parser state) to inject a synthetic `>` token for the subsequent closure evaluation.

4.4 Operator Precedence and Associativity

To eliminate structural ambiguity during the parsing of complex expressions, Cera enforces a strict, formalized hierarchy of operator precedence. The parser utilizes a Top-Down Operator Precedence (Pratt Parsing) architecture, assigning binding powers to individual operator tokens.

A critical architectural departure from legacy C-family languages is Cera's handling of bitwise operators. Historically, languages like C and Java assign bitwise AND, OR, and XOR a lower precedence than relational comparison operators, leading to non-intuitive evaluations (e.g., `a & b == c` evaluating as `a & (b == c)`). Cera corrects this by enforcing that all bitwise operations bind more tightly than equality and comparison checks, prioritizing mathematical correctness.

Operators are evaluated in the following order, from highest precedence (tightest binding) to lowest precedence:

1. **Call & Grouping:** `()`
2. **Unary Operators:** `!`, `~`, `-`
3. **Multiplicative (Factor):** `*`, `/`, `%` (Left-associative)
4. **Additive (Term):** `+`, `-` (Left-associative)
5. **List Concatenation:** `::` (Right-associative)
6. **Bitwise Shift:** `<<`, `>>` (Left-associative)
7. **Bitwise AND:** `&` (Left-associative)
8. **Bitwise XOR:** `^` (Left-associative)
9. **Bitwise OR:** `|` (Left-associative)
10. **Comparison:** `<`, `>`, `<=`, `>=` (Left-associative)
11. **Equality:** `==`, `!=` (Left-associative)
12. **Logical AND:** `&&` (Left-associative, Short-circuiting)
13. **Logical OR:** `||` (Left-associative, Short-circuiting)
14. **Conditional (Ternary):** `?  :` (Right-associative)

When operators of identical precedence appear within the same expression, their associativity dictates the order of evaluation. Left-associative operators are evaluated strictly from left to right. The *only* right-associative operators in the Cera language are list concatenation (`::`) and the ternary conditional (`?  :`), which evaluate from right to left, natively supporting recursive structural evaluation.

5 Abstract Syntax Tree (AST) Implementation

The translation of the parsed token stream into a logical hierarchy is governed by a purely immutable Abstract Syntax Tree (AST). The C# implementation completely eschews traditional,

deep object-oriented inheritance hierarchies in favor of data-centric structures that mirror Cera's own functional paradigms.

5.1 Node Architecture and Immutability

The AST is constructed exclusively using C# `record` types, guaranteeing that all nodes are deeply immutable upon creation. To rigidly categorize nodes and enforce compile-time safety without relying on behavioral inheritance, the compiler utilizes empty marker interfaces: `INodeAST`, `IExprAST`, `IStmtAST`, `ITypeAST`, and `IPatternAST`.

This architecture ensures that structural constraints (such as a `BinaryExpr` strictly requiring a left and right `IExprAST`) are enforced by the type system, while keeping the raw data representations entirely decoupled from compiler operations.

5.2 Pratt Parsing Mechanics

Expression evaluation utilizes a Top-Down Operator Precedence (Pratt) parsing architecture, orchestrated via an `InitializePrattParser()` registry. Rather than relying on deeply nested, rigid recursive descent grammar methods for every mathematical precedence level, the parser assigns semantic parsing delegates to specific token types dynamically:

- **Prefix Parsers:** Tokens that structurally initiate an expression (e.g., literals, identifiers, unary operators, or opening brackets) are mapped to a `PrefixParseFn` delegate.
- **Infix Parsers:** Tokens that operate between existing expressions (e.g., binary operators, the ternary question mark, or the function call parenthesis) are mapped to an `InfixParseFn` delegate, explicitly paired with their operational `Precedence` weighting.

During evaluation, the parser continuously polls the token stream for its assigned binding power. It greedily consumes tokens and recursively invokes their registered delegates for as long as the upcoming token's precedence strictly exceeds the current binding context, naturally producing a correctly ordered AST.

Overall we are implementing a Hybrid LL(1) & Pratt parser.

5.3 Tree Traversal via Pattern Matching

Because the AST nodes are implemented as pure data records devoid of behavioral logic, the compiler deliberately omits the traditional Object-Oriented Visitor pattern (i.e., nodes do not implement `Accept(IVisitor)` methods).

Instead, subsequent compiler phases—such as Semantic Analysis, Type Checking, and Bytecode Emission—traverse the AST utilizing modern C# functional pattern matching. By utilizing `switch` expressions over the base marker interfaces (like `IExprAST`), the compiler can cleanly and exhaustively unwrap node structures. This centralizes pipeline logic within the respective compiler phases, avoids interface bloat, and aligns the compiler's codebase with the functional design philosophy of the Cera language itself.

6 Semantic Analysis

Context-sensitive validation checks, including scoping, constant resolution, type checking, and type inference rules.

7 Intermediate Representation (IR)

Lowering high-level functional elements from the AST into a structured linear or functional intermediate form (e.g., A-Normal Form or direct 3-Address Code) optimized for emission.

8 Bytecode Emission

Serialization rules mapping the optimized IR into binary operational codes (opcodes). Detail your custom Instruction Set Architecture (ISA) here, including any specialized register encoding or branch features.

9 The Virtual Machine (VM)

The runtime environment executing the bytecode via a continuous Fetch-Decode-Execute cycle. Define the structure of your virtual register array, evaluation stacks, and runtime memory layouts.

10 Planned Updates

10.1 Excluded Features

Before considering future features for the language, we shall consider previous planned additions, that where decided against for various reasons:

- **Lazy Keyword, Force Pattern:** When the language was first being developed, the idea of a `lazy` keyword was planned, allowing for delayed evaluation thunks similar to unit thunks, but caching the results upon evaluation. This aforementioned similarity, was the primary reason for it's exclusion (see design principles), as well as adding a vast, complicated system, that can be replicated without the use of unique keywords and syntax.

10.2 Planned Features

Now we may take an optimistic look into the future of the language:

- **Inline Keyword:** Very similar to it's implementation among OOP languages, inline functions will simply insert their body directly in place of it's call, rather than requiring an extra frame to be placed onto the stack. While a feature that will be added, omitted for now to reduce initial complexity. e.g. `def inline double(x : float) : float = { 2 * x; }`
- **Top Level Var Declarations:** This can be extremely useful feature, replacing features such as the currently rather clunky format of constants `def pi() : float = 3.14159265358979;` , and has been omitted from the first version of Cera purely for garbage collector and passing complexity reduction.
- **Switch If Statements:** An embedded way for switch statements to use ifs, similar to OCaml's when keyword, aswell as the removal of the requirement of braces for if statements.
- **Default Function Values:** Similar to C#, with `param : type = value`.
- **Inner Named Functions:** As the name states.